

REVISION TWO • THE ARCHITECTURE, CORRECTED

A NEXT GENT

Not one system with one central flow. A federation of independent systems that share standards, identity, data contracts, execution methods and trust.

SYSTEMS

Fifteen

LEVELS

Vision / Platform / Wedge

PROVING GROUND

Alabama Gulf Coast

EXECUTION

Device-first

CONTENTS

- 01 What changed, and why it matters
 - 02 Three levels, kept separate
 - 03 The fifteen systems
 - 04 What independence actually means
 - 05 A business is a graph, not a record
 - 06 Common centre, vertical edges
 - 07 The learning flywheel
 - 08 Two dimensions of state
 - 09 CyberCheck: consistency, not confirmation
 - 10 Customer capture and reconciliation
 - 11 The cancellation loop
 - 12 Independence is not build order
 - 13 The order
 - 14 The wedge
-

What changed, and why it matters

Every earlier description of this project made the same error, and it was not an error of omission. The pieces were mostly right. They were arranged inside the wrong mental model.

The wrong model was: *one system, one central flow, one object passing through five stages*. It is a tidy way to describe software and it is the natural instinct of anyone trying to make a complicated thing legible. It is also false here, and following it produces bad decisions — a single build order, a single interface, a single point where everything must integrate before anything works.

The correct model is a **federation**. Fifteen systems that each do something useful alone, that share standards, identity, data contracts, execution methods and trust mechanisms when connected, and that become considerably more powerful together than apart. Some customers will use one. Some will connect several. None of them has to wait for all of it.

The practical consequence: there is no big-bang integration, no single release where the system starts working, and no part that is useless until the rest exists.
Each system has its own reason to be built and its own customer.

Three levels, kept separate

The second correction is about scope, and keeping these three apart is what lets the ambition stay large while the work stays finishable.

LEVEL ONE

The vision

A modular, device-first computing and application ecosystem. Home, family, business, consumer, local search, commerce, hardware, developer economy, edge. This is legitimately where the architecture can go, and describing it honestly is not overreach — it is what makes the platform decisions below it correct rather than accidental.

LEVEL TWO

The platform

Independent systems for execution, structured business data, automation, trust, applications, discovery and customer relationships. This is the part with an architecture, a data model and a build order. Most of this document lives here.

LEVEL THREE

The wedge

One real business on the Alabama Gulf Coast. Its actual phone, its actual applications, its actual bookings, its actual availability, its actual confirmation email, its actual cancellation, its actual Google Business profile. Real-time availability, customer capture, one-update-everywhere and verified execution.

Keeping them separate prevents two opposite failures. It stops the vision from being quietly shrunk to whatever ships first, and it stops the wedge from being delayed until a diagram containing automotive and family computing is finished.

If the wedge works, the enormous architecture has a foundation. If it does not, **a beautiful diagram containing an App Store and an edge strategy does not matter.**

The fifteen systems

Each one stands alone. Each one is stronger connected. The numbering is a reference, not a sequence — they are not built in this order and several are built at once.

01 Device & execution Android handset, Linux node, browser, containers, local and cloud compute. UI automation, deterministic executors, human takeover, shared application maps.	02 Automation & agent Intent to planner to known workflow or exploration, approval where required, execution, verification. AI handles uncertainty; deterministic code handles learned work.
03 Structured business data Hierarchical entities, industry-neutral core, vertical-specific tables, provenance, observations, availability, menus, events, pricing, rules.	04 CyberCheck trust Provenance, identity, last verification, source, device, actor, contradictions, freshness. Proof that a fact or an action is what the system claims.
05 Public profile Composable sections projected from structured data. Many visual presentations, one underlying truth, no duplication.	06 Availability & booking intelligence Email parsing, iCal, CSV, manual inventory, APIs where practical. Normalised availability, booking reconciliation, last-minute inventory.
07 Customer relationship Capture before redirect, reconcile after confirmation, CRM, SMS, loyalty, rewards, review requests, retargeting.	08 Review Verified customer reviews, plus the owner reply centre with AI-assisted drafting, approval, and a voice that learns.
09 Discovery & directory Local structured search over the business hierarchy and real operational data rather than flat listings.	10 App Store A genuinely independent marketplace for installable capabilities. Businesses assemble their own system from modules.
11 Third-party applications Google Business, Facebook, Toast, FareHarbor, Peek. Not App Store modules — software the environment interacts with.	12 Developer platform SDK, MCP, skills, scripts, workflows, templates, app publishing, reusable execution maps.
13 VAPPS	14 Interface

Voice-native and assistant-native applications exposing the same capabilities without a conventional graphical app.	TV, desktop, tablet, phone, kiosk, dashboard, consumer. Projections of the systems underneath, not the architecture itself.
15 Infrastructure Isolated customer containers, persistent state, idle and wake economics, cloud bursting, updates, backups, orchestration.	

Two entries deserve emphasis because getting them wrong causes category errors elsewhere.

Eleven is not ten. Toast and Facebook are third-party software the environment operates. They are not modules in your marketplace and they never will be. Confusing the two produces a roadmap full of integrations that were never yours to build.

Fourteen is not the architecture. The smart-TV layout is one projection. A phone, a tablet, a desktop and a kiosk are others. Screen size should not change the product, and no earlier description of this should have led with an interface.

What independence actually means

Independence is a claim about coupling, and it is worth being precise about what it does and does not promise.

It means

- A system can be deployed on its own, for a customer who wants only that.
- Another system failing degrades it rather than stopping it.
- It can be replaced with a different implementation behind the same contract.
- It can be sold separately, and some of them should be.

It does not mean

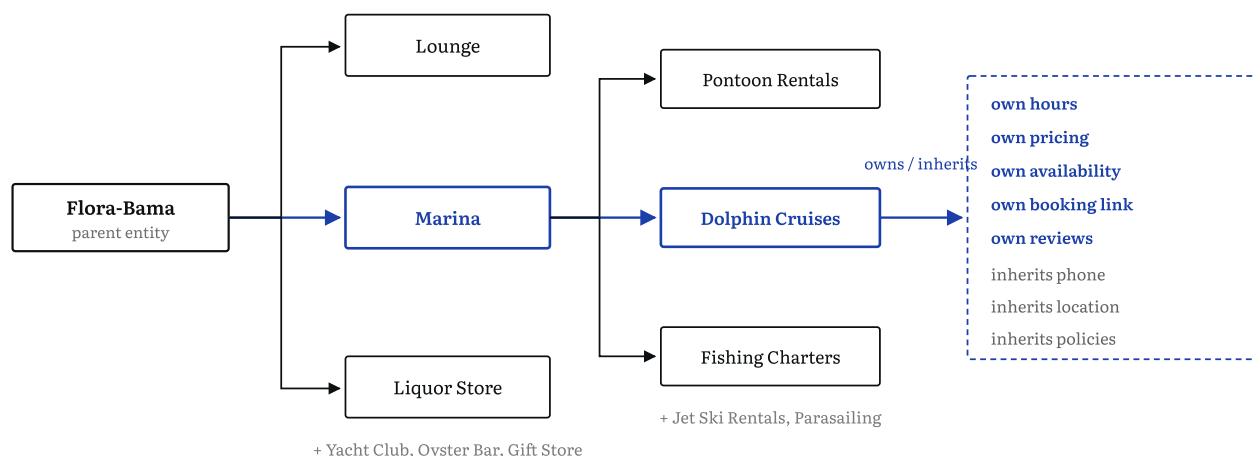
- They can all be built in parallel by a small team. Independence is architectural; capacity is not.
- They are equally valuable at the start. An App Store with nothing in it has no customer.
- They share nothing. They share exactly five things, and those five are the federation.

The five shared things: **identity** (who is asking), **data contracts** (what a business fact looks like), **execution methods** (how an action is carried out), **trust mechanisms** (how a claim is proven), and **standards** (MCP, the manifest, the capability names). Everything else is local to a system.

A business is a graph, not a record

This belongs at the beginning of the data model, not in a later chapter about edge cases, because almost every bad assumption downstream comes from treating a business as one flat row.

A real Gulf Coast business is an entity graph with **independently addressable operating units**. Flora-Bama is not a restaurant. It is a lounge, a yacht club, an oyster bar, a marina, a liquor store and a gift store — and the marina is itself pontoon rentals, jet ski rentals, dolphin cruises, parasailing and fishing charters.



Every node in the graph is independently addressable. Dolphin Cruises has its own hours, prices, availability, booking link and reviews, while inheriting the phone number, location and policies from the marina above it. **A data model that assumes one business is one record cannot represent this** — and most businesses worth selling to are shaped this way.

Each node independently owns or inherits identity, location, hours, phone, menus, products, pricing, rules, policies, events, inventory, availability, booking links, reviews, media, specials and amenities. Inheritance is the default; ownership is the override.

Getting this right early is cheap. Retrofitting it after a hundred businesses are in a flat schema is a migration nobody enjoys, and it is the single most likely reason an otherwise sound data layer has to be rebuilt.

Common centre, vertical edges

Underneath the graph sits the table-per-section philosophy: every meaningful section of a business gets its own table. No giant JSON blob pretending a restaurant and a marina are the same shape with different keys.

The boundary that makes this work without building a separate platform per industry: **common primitives in the centre, vertical operational facts at the edges.**

Shared by everyone

entities	locations	hours	contacts
customers	reviews	events	media
permissions	communications	automations	

Diverging by vertical

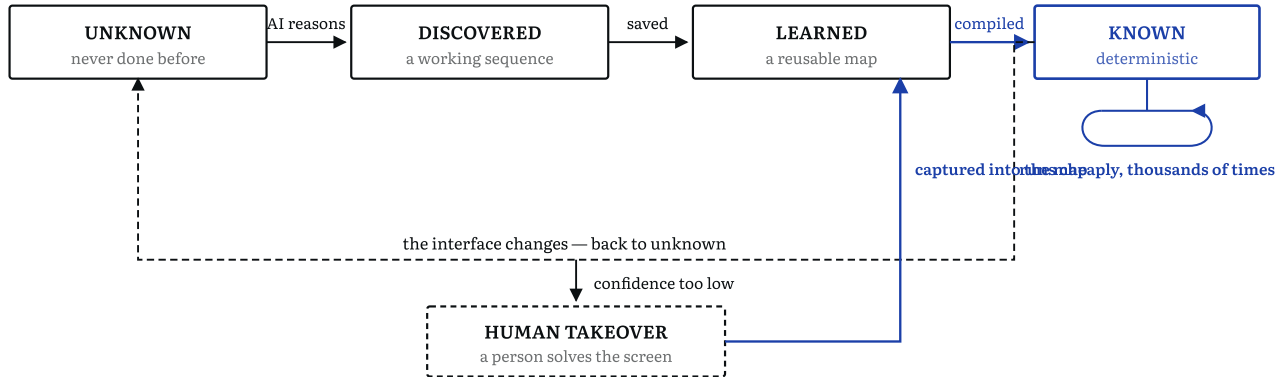
RESTAURANT	CHARTER / TOURISM	MARINA
menus	charter_trips	slips
menu_sections	start_times	vessels
menu_items	capacity	rental_inventory
happy_hour	availability	dock_services
tables	pricing_tiers	
	waivers	ELECTRICIAN
	deposits	service_areas
	addons	service_types
		quote_requests
		emergency_calls
		appointments
		job_photos

One business slug connects to whichever tables apply. A marina that also runs a kitchen uses both sets. Nothing forces an industry into fields that do not fit it, and nothing requires a new platform to add one.

This is what makes a vertical cheap. Adding electricians means adding six tables and a set of application maps — **not a new product**. Identity, customers, reviews, communications and automations are already there.

The learning flywheel

Calling the agent "deliberately dumb" was inaccurate shorthand and it hid the mechanism that actually matters. AI is the **planner**, not the clicker. The system continuously converts expensive reasoning into cheap deterministic execution.



Reason once, execute cheaply many times. The dashed return path is what an app redesign triggers; the branch below it is what happens when the system is not confident enough to continue. **Human takeover is not only a fallback — it is part of the learning architecture**, because the solution a person finds becomes a permanent improvement to the shared map.

The economics are the point. Frontier reasoning costs real money per screen. Replaying a known sequence costs effectively nothing. A system that converts the first into the second every time it meets something new gets cheaper per business as it grows, instead of more expensive.

And the map is shared. One business hits Toast's redesign; every business gets the fix. That is the same flywheel described from the customer's side.

Two dimensions of state

An action has a lifecycle. Its result has an outcome. Collapsing those into one status field is how silent partial success gets reported as done.

LIFECYCLE — WHERE IT IS

REQUESTED

PLANNED

AWAITING_APPROVAL

EXECUTING

VERIFYING

FINISHED

VERIFICATION — WHAT HAPPENED

VERIFIED

PARTIALLY_VERIFIED

NOT_YET_VERIFIABLE

CONTRADICTED

FAILED

A price change to \$349 runs. Website: \$349. Profile: \$349. Google: \$349. Facebook: still \$399. The lifecycle is **FINISHED**. The verification is **PARTIALLY_VERIFIED**, three of four surfaces consistent, Facebook unresolved. Neither field alone can say that, and a single status field would have said "completed."

Silent partial success is more dangerous than clean failure. A failure tells you nothing changed. A partial success creates two competing realities, and the business does not know which one a customer is looking at.

Which means every capability also has to declare what to do about it: roll the three back, or drive the fourth forward. One or the other, decided per capability, written into the manifest rather than improvised at three in the afternoon.

CyberCheck: consistency, not confirmation

The weak version of a trust layer asks: *did the automation click Save?* That is a question about a button. The strong version asks four questions about the world: **what is currently true, how do we know it, where is it inconsistent, and when did we last prove it?**

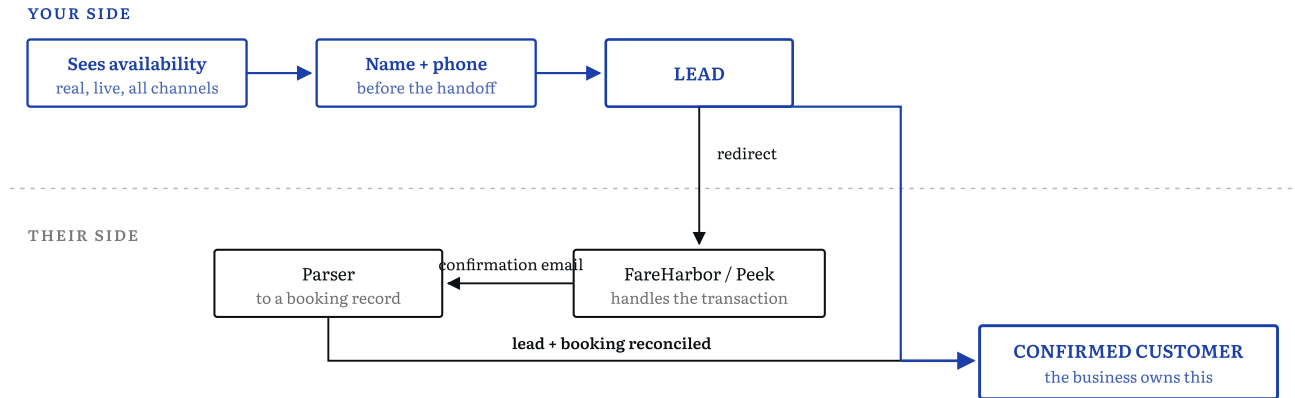
CANONICAL PRICE — \$349		
A NEXT GENT profile	VERIFIED	\$349
Business website	VERIFIED	\$349
Google Business	VERIFIED	\$349
Facebook	CONTRADICTED	\$399
Yelp	NOT_YET_VERIFIABLE	

That table is a product. It is the thing a business owner cannot get anywhere at any price today, and it is the thing an insurer, a franchisor or a chamber would recognise immediately as valuable.

It also gives the trust layer its real job list: provenance, identity, last verification, source, device, actor, contradictions and freshness. Not more data — **provable data**, with a record of who changed what, from which device, and when it was last confirmed against reality.

Customer capture and reconciliation

This is a core business mechanism, not a feature of booking, and it deserves to be named at the top level. It is how a business takes back the thing every gatekeeper platform is designed to keep: **the customer relationship**.



The provider still takes the transaction, and nothing asks the business to leave a platform that works. But the visitor was identified **before** the handoff, and the confirmation email closes the loop afterwards — so the anonymous click that a gatekeeper normally keeps becomes a named customer in the business's own records.

Then the lifecycle continues, and every stage is something the business could never do before: booking, experience, verified customer, review request, loyalty, future offer, repeat booking. None of that is reachable if the customer only ever existed inside someone else's platform.

The cancellation loop

Availability data becomes a revenue engine rather than a nicer calendar at exactly one moment: when something gets cancelled.

```
10 AM dolphin cruise
capacity    20
booked      20
cancelled    3
available    3

      ↓

find customers who
  searched that trip
  requested that time
  joined the waiting list
  previously booked something similar

      ↓

SMS - "3 seats just opened tomorrow at 10."
```

Every input to that query already exists because of the systems above it. The availability came from the parser. The customers came from capture and reconciliation. The message goes out on the business's own number. **Nothing new has to be built; the pieces connect.**

And the money is real. That inventory was going to expire worthless in eighteen hours. This is the clearest demonstration in the whole product of why the systems are worth more connected than apart — and it is a very good thing to show an owner in a first meeting.

Independence is not build order

Two statements that sound similar and are not.

Architecturally, the App Store is an independent subsystem. True. It has its own contracts, it can be deployed alone, other systems work without it, and an application from it could be used by someone who bought nothing else.

Therefore it can be built early. False. An empty store has no customer, and a store is worth exactly what is installable in it.

The real progression runs through the product, not around it:

- 1 A real business problem worth money.
- 2 Availability and structured data that solve it.
- 3 One update reaching everywhere, verified.
- 4 Customer capture and ownership.
- 5 **Actual paying businesses.**
- 6 Reusable first-party modules emerge from serving them.
- 7 Those modules get packaged as apps.
- 8 Now the App Store has something worth installing.
- 9 The SDK, once there is a pattern worth documenting.
- 10 Third-party marketplace.

The store is not built late because it is unimportant. It is built late because **its first inventory is the first-party modules that only exist after real businesses have been served**. Building it first means designing a marketplace for applications nobody has written for customers nobody has met.

The order

Within the federation, some systems are load-bearing for the wedge and some are not. This is the sequence for the parts that are — and each step produces something a business would pay for before the next one starts.

- 1 **Prove the executor still works.** Real applications an Orange Beach charter actually runs, on a real handset. One afternoon. Everything else assumes this.

- 2 **One capability, end to end.** Change a real business's hours, verify it on a different path, record the result with both dimensions of state.

- 3 **The entity graph and the neutral core.** Hierarchy, provenance, observations. Before the first email is parsed, not after.

- 4 **The email parser.** Bookings, cancellations, sales summaries, normalised into availability.

- 5 **Public availability.** On the business's own page. This is the first thing that earns the fee by itself.

- 6 **Capture and reconciliation.** Lead before redirect, booking after, joined into a customer the business owns.

- 7 **SMS on the real number.** Approvals, review requests, and the cancellation loop.

- 8 **The shared map pipeline.** Nightly checks, one re-recording, signed, pushed to everyone. Now the subscription is structurally real rather than a promise.

- 9 **CyberCheck consistency.** Canonical versus every surface, with freshness.

- 10 **Modules, then the store, then the SDK.** In that order, for the reason in the previous chapter.

Systems not on this list — the consumer layer, VAPPS, the developer marketplace, the appliance OS itself — are not deferred because they are wrong. They are deferred because **none of them is what makes the first twenty-five businesses pay.**

Note what is absent from steps one through seven: no operating system, no appliance, no hardware, no inventory, no app store. **The wedge ships as software on machines that already exist**, which is what makes finding out you are wrong cost a month instead of a year.

The wedge

One real business on the Alabama Gulf Coast. Orange Beach, Gulf Shores, Perdido Key, Fort Morgan, Foley, Lillian — a small enough area to reach in a day and dense enough that twenty businesses in it is a market rather than a scatter.

What has to be proven is one sentence:

A NEXT GENT can observe reality, normalise it, act on it, verify the result, and create measurable economic value.

Each clause is testable on its own, and in this order:

- **Observe reality.** The confirmation email arrives and the parser reads it correctly.
Measurable: does the count match what the owner says is true?

- **Normalise it.** Three platforms with three formats become one availability model.
Measurable: does it reconcile against the operator's own board at the end of a day?

- **Act on it.** Change something in the real world through the real applications.
Measurable: did the screen change?

- **Verify the result.** On a different path than the one that wrote it. Measurable: do two independent reads agree?

- **Create economic value.** Bookings that moved direct, seats sold from a cancellation, hours that were right when someone checked. Measurable in dollars, from their own statements.

That last clause is the only one that matters to the customer, and the first four exist to make it repeatable rather than lucky.

The one thing still unrun

Install the applications an Orange Beach charter operator actually uses. Turn on an accessibility service. Watch what still runs. Some categories of application detect it and refuse to operate, and payment and point-of-sale software is precisely the open question — and precisely the software that matters most here.

It costs an afternoon and a forty-dollar phone. **Nothing above it is worth starting until it is answered**, and if the answer is bad it is far better to know now, with a phone, than in month nine with hardware ordered and customers signed.

What this revision changed.

Not a feature. The mental model. Earlier descriptions put the right pieces inside the wrong frame — one system, one flow, one build order, one interface — and that frame produced a linear plan for something that is not linear.

What replaced it: **a federation of fifteen independent systems, three levels kept deliberately separate, a business modelled as an entity graph rather than a record, and two dimensions of state instead of one.** Everything that was true before is still true. It is arranged correctly now.

And the thing that has not changed across any revision: it still comes down to one afternoon, one phone, and one Orange Beach charter operator's real applications.